Blockchain Lab Exp 4

Name: Chirag Choudhary **Class:** D20A
Roll no: 17 **Batch:** A

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions (pure, view)

⍰ Primitive Data Types

Primitive data types are the basic building blocks used to store simple values in Solidity.

- **uint (Unsigned Integer)**
  This represents non-negative whole numbers (0, 1, 2…). It is commonly used for balances, counters, and indexes. Solidity provides different sizes like uint8, uint256 (default). Larger sizes consume more storage but prevent overflow.
- **int (Signed Integer)**
  Used to store both positive and negative numbers. Rarely used compared to uint because blockchain values are usually non-negative.
- **bool (Boolean)**
  Stores either true or false. It is typically used in conditions like checking whether a task is completed.
- **address**
  Stores Ethereum account or contract addresses. It is very important because it represents users or smart contracts interacting with the blockchain.
- **bytes**
  Used to store raw binary data. Fixed-size (bytes1 to bytes32) and dynamic-size (bytes) are available.

⍰Variables

Variables are used to store data in a smart contract.

- **State Variables**
  Declared inside a contract but outside functions. Stored permanently on the blockchain. Example: storing balances or user data.
- **Local Variables**
  Declared inside functions. Temporary and only exist during function execution.
- **Global Variables**
  Provided by Solidity automatically, like msg.sender (caller's address) or block.timestamp.

Functions (pure and view)

Functions define the behavior of the contract.

- **view functions**
  These functions only read data from the blockchain but do not modify it. They do not cost gas when called externally. Example: checking a stored value.
- **pure functions**
  These functions neither read nor modify blockchain data. They only perform calculations based on inputs. Example: adding two numbers.

Difference:

- view → reads state
- pure → does not read or write state

2. Inputs and Outputs to Functions

Functions can accept inputs (parameters) and return outputs.

- **Inputs (Parameters)**
  These are values passed to a function when calling it. They allow dynamic behavior instead of fixed logic.
- **Outputs (Return Values)**
  Functions can return one or multiple values. These outputs help retrieve processed data from the contract.
- Functions can be:

  Without input

  Without output

  With both input and output

⍰ Important:
In blockchain, inputs are part of transaction data, and outputs are returned to the caller.

3. Visibility, Modifiers, and Constructors

⍰ Visibility

Visibility defines who can access functions or variables.

- **public**
  Accessible from anywhere (inside and outside the contract).
- **private**
  Accessible only inside the contract where it is defined.
- **internal**
  Accessible inside the contract and derived contracts.
- **external**
  Can only be called from outside the contract (not internally).

⍰ Modifiers

Modifiers are used to add conditions or restrictions to functions.

For example, you can restrict a function so that only the owner can call it. Instead of writing the same condition repeatedly, modifiers help reuse logic.

They improve:

- Code readability
- Security
- Reusability

⍰ Constructors

A constructor is a special function that runs only once when the contract is deployed.

- Used to initialize values
- Often used to set the contract owner

Once deployed, the constructor cannot be called again.

4. Control Flow: if-else, loops

⍰ if-else

Used for decision-making.

- If condition is true → execute block
- Else → execute alternative block

It is commonly used in validation, like checking balances or permissions.

⍰ Loops

- **for loop**
  Repeats execution a fixed number of times.
- **while loop**
  Runs as long as a condition is true.

⍰ Important Note:
Loops should be used carefully in Solidity because:

- They consume gas
- Large loops can cause transaction failure

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.

- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

Tutorial 1: Introduction

1) Compiling and deploying-

REMIX 2.0.0

Learneth Solidity Beginner Course

Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

Deployed Contracts 1 — + Add Contract

Interact with a deployed contract

Counter — 0xaE0...96B8b — Remix VM — 6min ago

Transactions recorder 1 — Save

Contract call | Transaction list — Newest

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x74f...c98d5 — Debug

REMIX 2.0.0

Learneth Solidity Beginner Course

Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

Low level interaction

call get

0:uint256: 0

Call

Balance — 0 ETH

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x74f...c98d5 — Debug

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c — Debug

REMIX 2.0.0

Learneth Solidity Beginner Course

Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

call get

Low level interaction

call get

0:uint256: 3

Call

Balance — 0 ETH

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xaE0...96B8b value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xa85...9d1ea — Debug

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c — Debug

remix.ethereum.org/?#activate=udapp.solidity.LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.0.0 | Learneth Solidity Beginner Course | Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

non-payable **dec**

Value: 0 wei

Gas limit: auto 0

Transact

Balance: 0 ETH

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c — Debug

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xaE0...96B8b value: 0 wei data: 0xb3b...cfa82 logs: 0
hash: 0xb2b...6c83e — Debug

remix.ethereum.org/?#activate=udapp.solidity.LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.0.0 | Learneth Solidity Beginner Course | Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

call **count**

call **get**

Low level interaction

call **get**

0:uint256: 2

Call

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xaE0...96B8b value: 0 wei data: 0xb3b...cfa82 logs: 0
hash: 0xb2b...6c83e — Debug

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c — Debug

REMIX 2.0.0 | Learneth Solidity Beginner Course | Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM — Osaka

Account 1 — 0x5B3...eddC4 — 99.999 ETH

call **count**

call **get**

Low level interaction

call **count**

0:uint256: 2

Call

Compile | introduction.sol | basicSyntax.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Counter {
    uint public count;

    // Function to get the current count
    function get() public view returns (uint) {    2453 gas
        return count;
    }

    // Function to increment count by 1
    function inc() public {    infinite gas
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {    infinite gas
        count -= 1;
    }
}
```

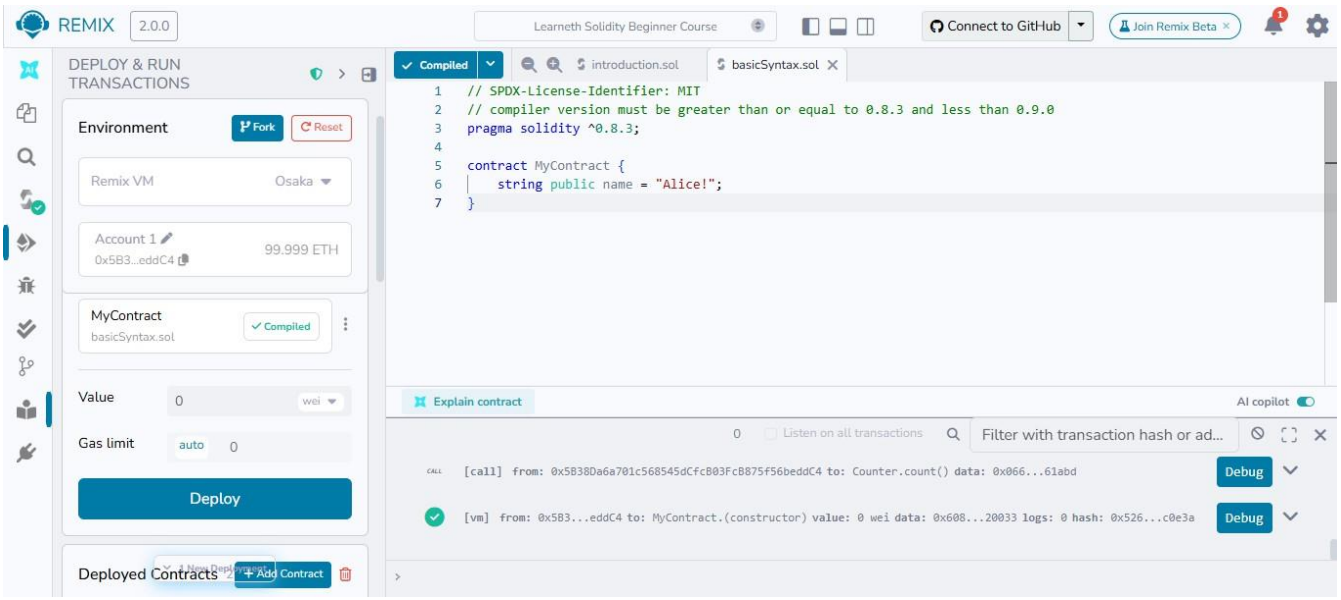
Explain contract | AI copilot

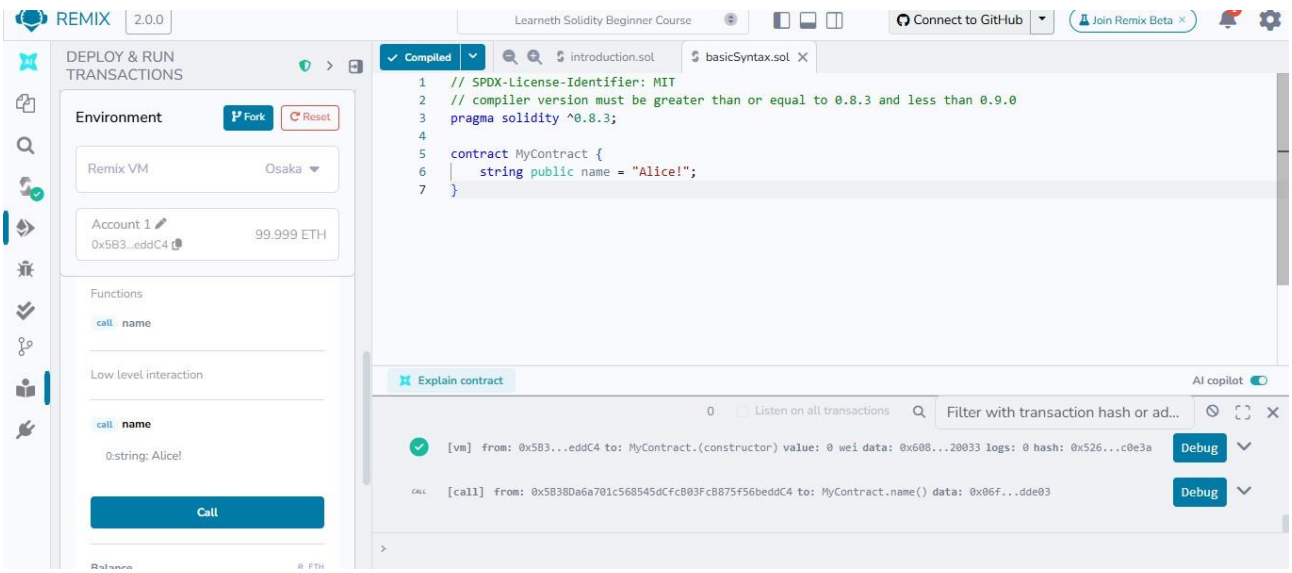
0 | Listen on all transactions | Filter with transaction hash or ad...

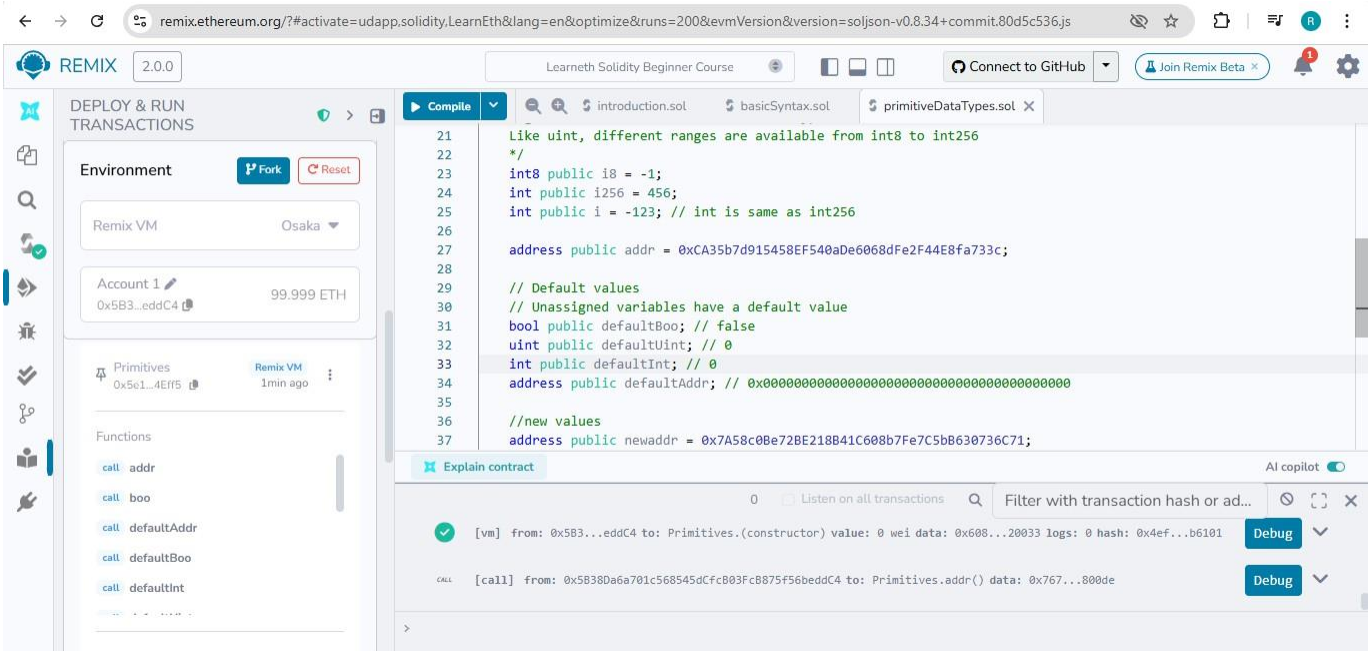
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c — Debug

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd — Debug

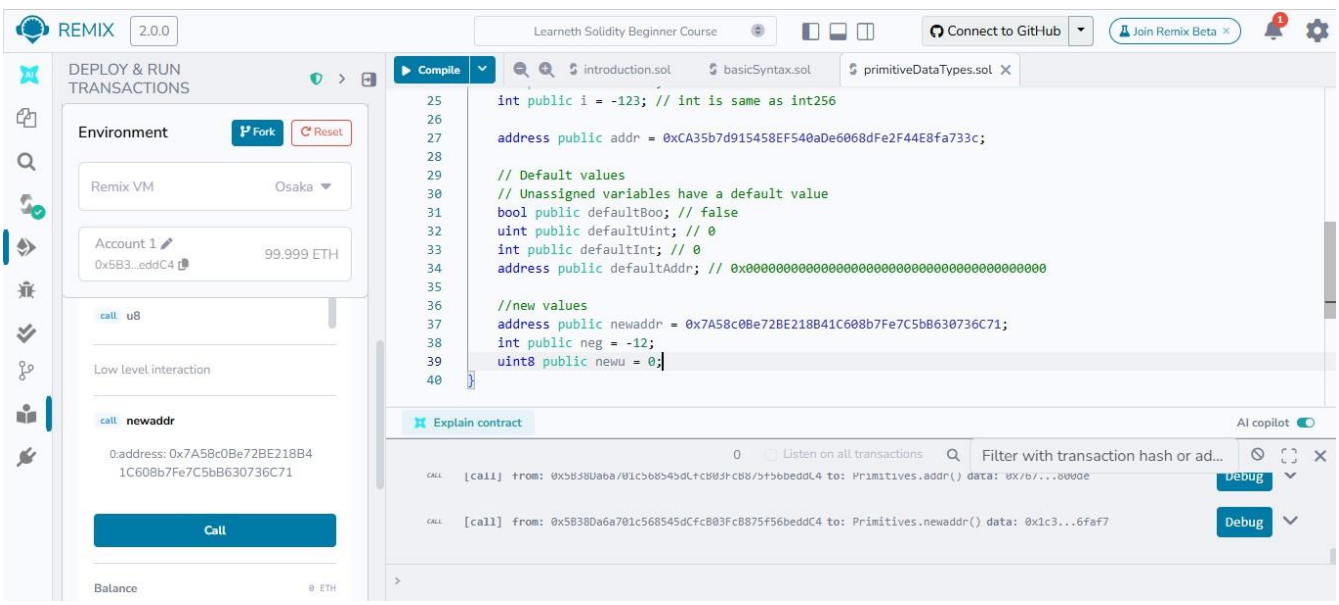
Tutorial 2: Basic Syntax







Tutorial 3: Primitives



remix.ethereum.org/?#activate=udapp,solidity,LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.0.0 | Learneth Solidity Beginner Course | Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM: Osaka

Account 1 0x5B3...eddC4: 99.999 ETH

Low level interaction

call neg

0:int256: -12

Call

Balance: 0 ETH

Compile | introduction.sol | basicSyntax.sol | primitiveDataTypes.sol

```
    int public i = -123; // int is same as int256

    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;

    // Default values
    // Unassigned variables have a default value
    bool public defaultBoo; // false
    uint public defaultUint; // 0
    int public defaultInt; // 0
    address public defaultAddr; // 0x0000000000000000000000000000000000000000

    //new values
    address public newaddr = 0x7A58c0Be72BE218B41C608b7Fe7C5bB630736C71;
    int public neg = -12;
    uint8 public newu = 0;
}
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Primitives.newaddr() data: 0x1c3...6fa7 Debug

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Primitives.neg() data: 0x780...63b1f Debug

Scam Alert | Initialize as git repo | Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. | RemixAI Copilot (enabled)

remix.ethereum.org/?#activate=udapp,solidity,LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5c536.js

REMIX 2.0.0 | Learneth Solidity Beginner Course | Connect to GitHub | Join Remix Beta

DEPLOY & RUN TRANSACTIONS

Environment: Fork | Reset

Remix VM: Osaka

Account 1 0x5B3...eddC4: 99.999 ETH

call u256

Low level interaction

call newu

0:uint8: 0

Call

Balance: 0 ETH

Compile | introduction.sol | basicSyntax.sol | primitiveDataTypes.sol

```
    int public i = -123; // int is same as int256

    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;

    // Default values
    // Unassigned variables have a default value
    bool public defaultBoo; // false
    uint public defaultUint; // 0
    int public defaultInt; // 0
    address public defaultAddr; // 0x0000000000000000000000000000000000000000

    //new values
    address public newaddr = 0x7A58c0Be72BE218B41C608b7Fe7C5bB630736C71;
    int public neg = -12;
    uint8 public newu = 0;
}
```

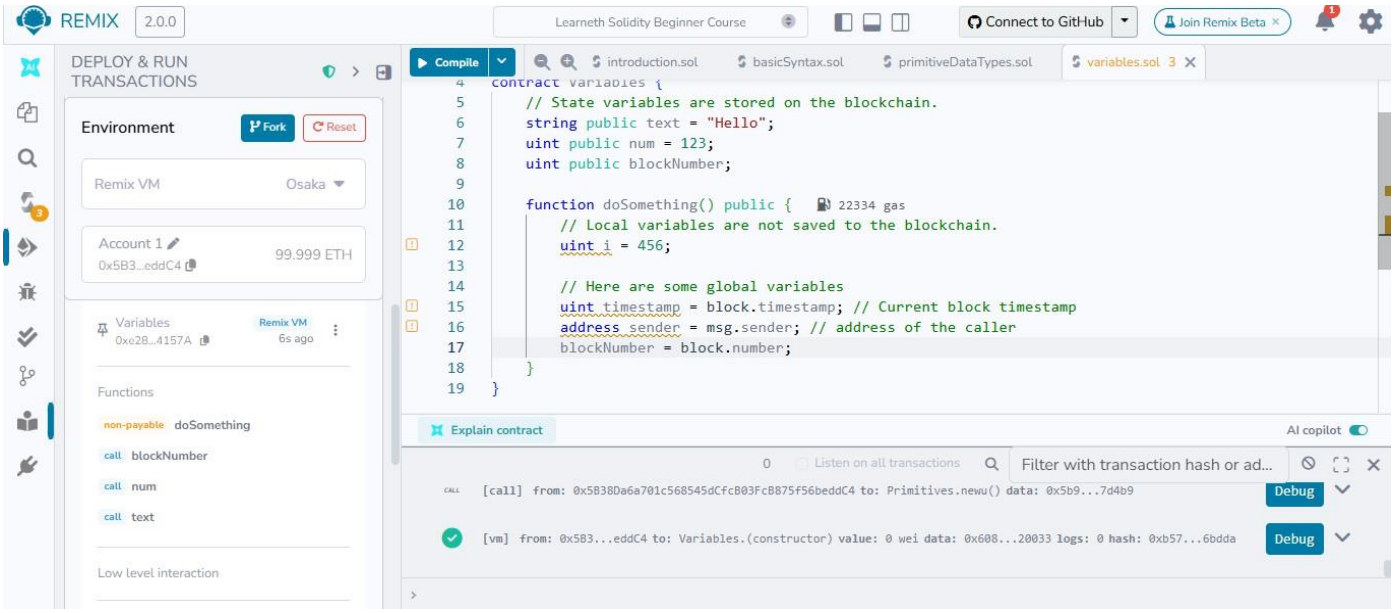
Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

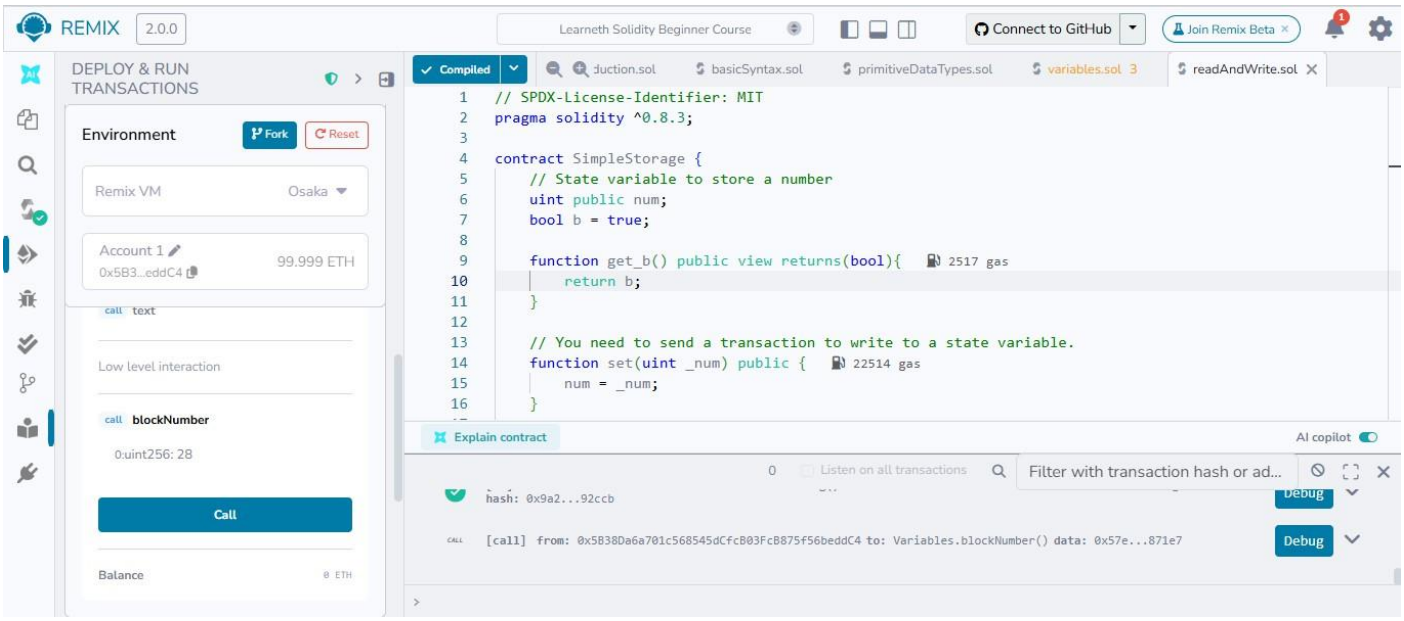
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Primitives.neg() data: 0x780...63b1f Debug

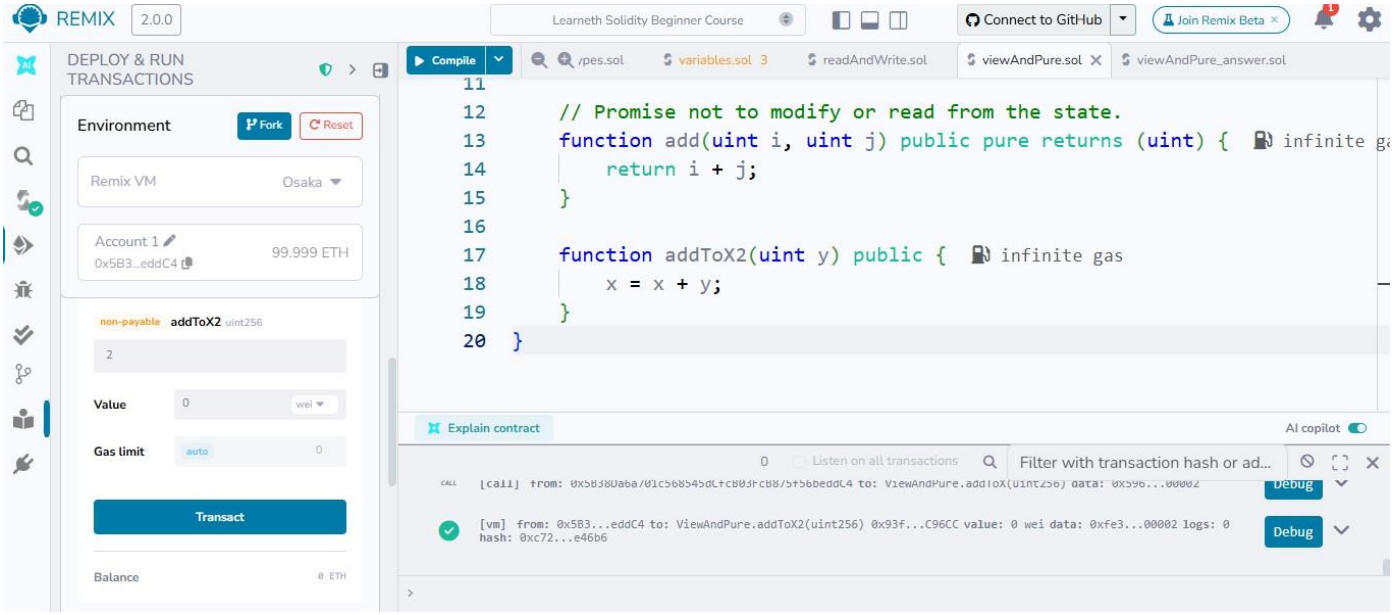
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Primitives.newu() data: 0x5b9...7d4b9 Debug

Tutorial 4. Variables

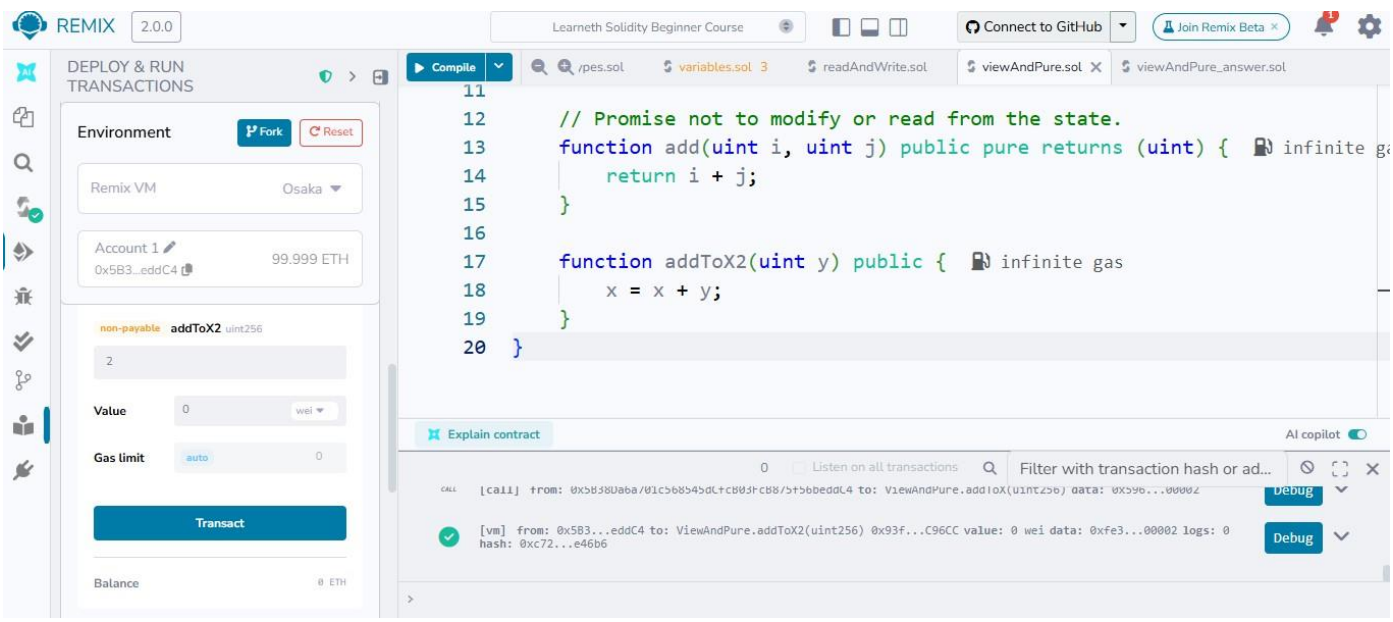


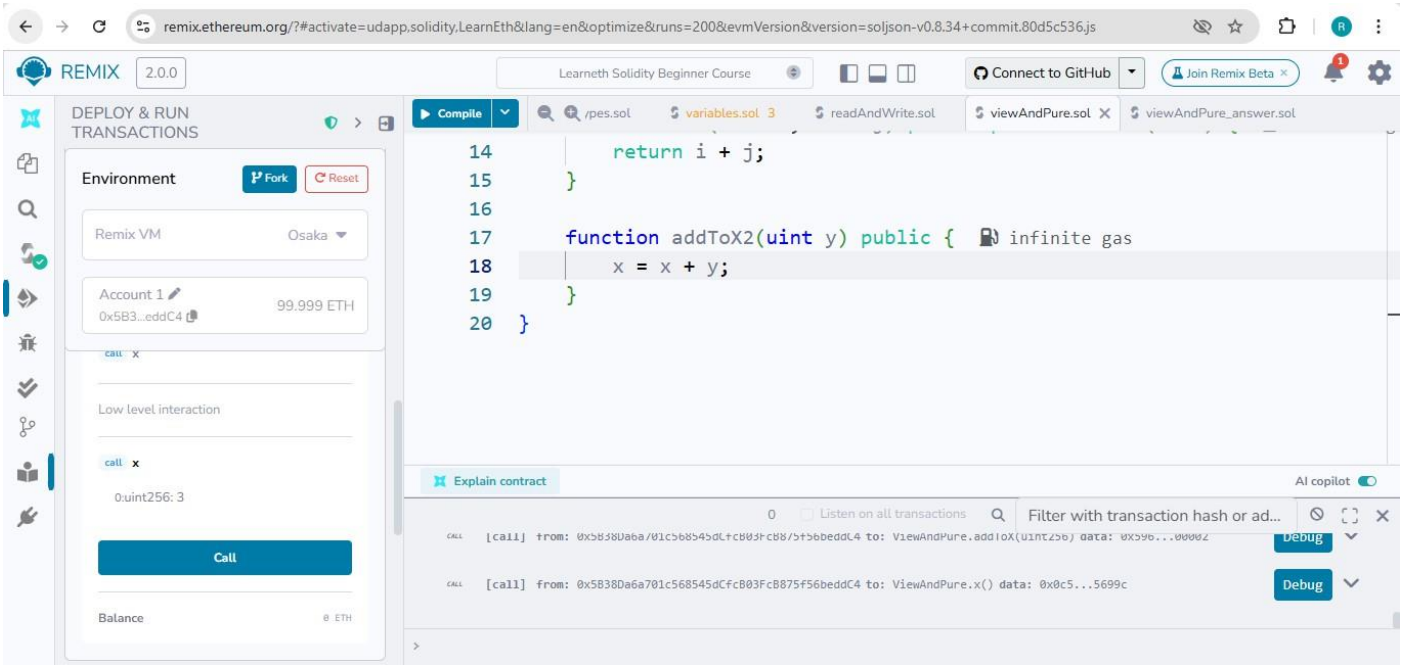
Tutorial 5.1 Functions - Reading and Writing to a State Variable



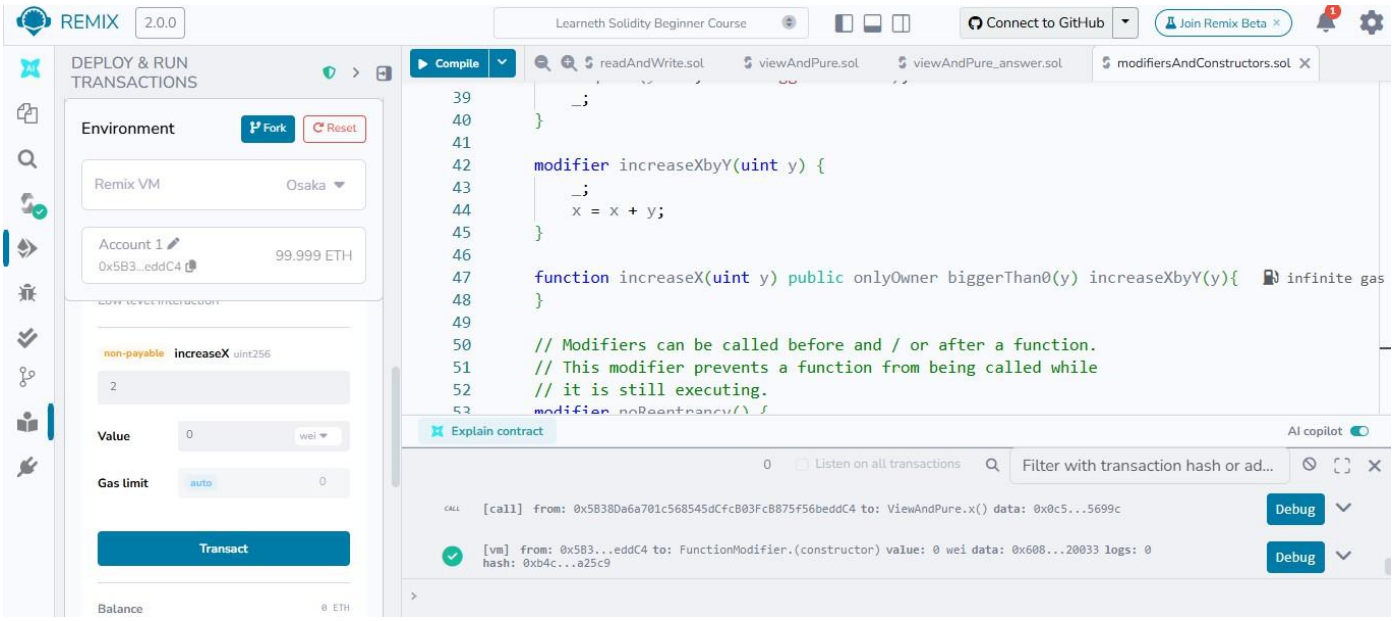


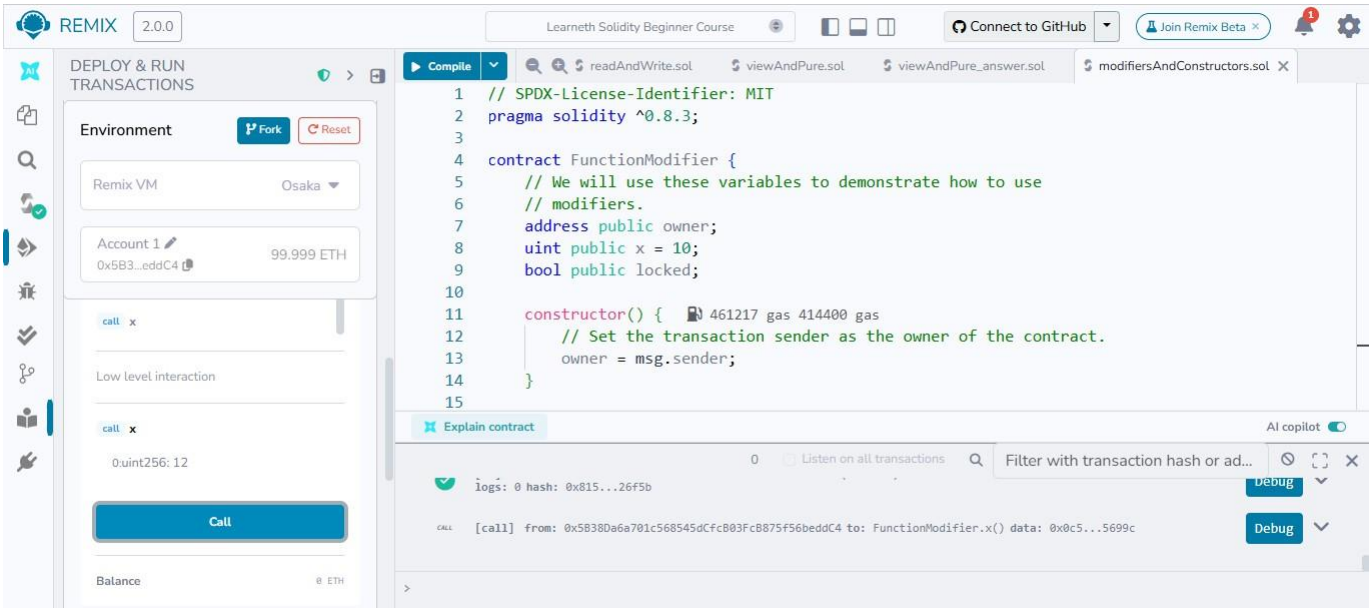
Tutorial 5.2 Functions - View and Pure



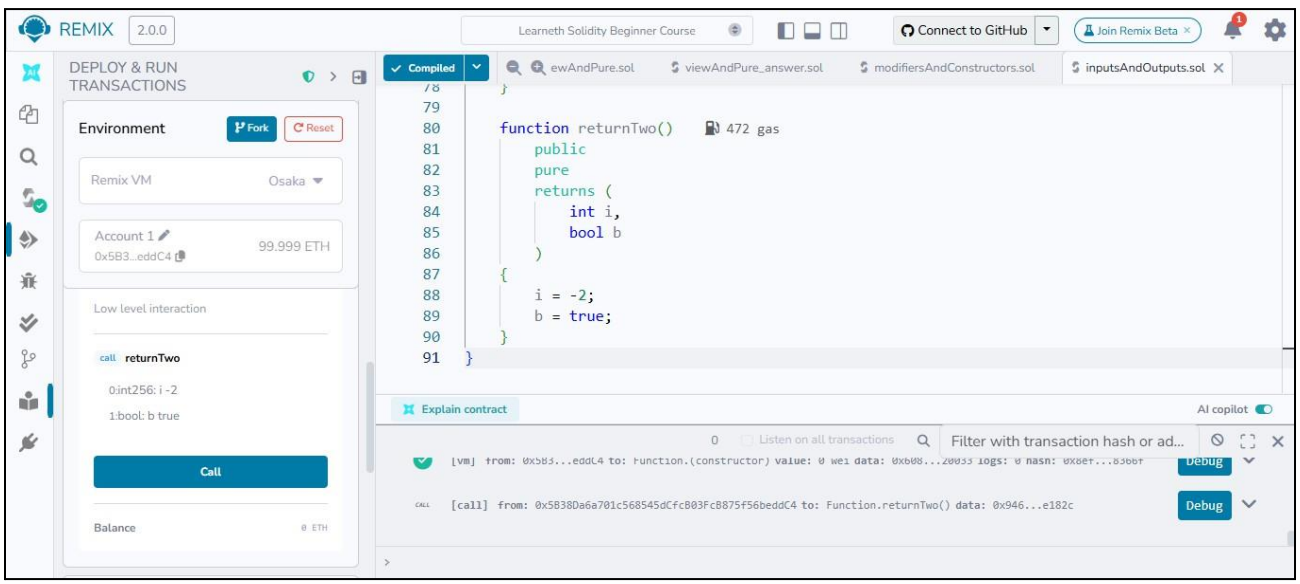


Tutorial 5.3 Functions - Modifiers and Constructors

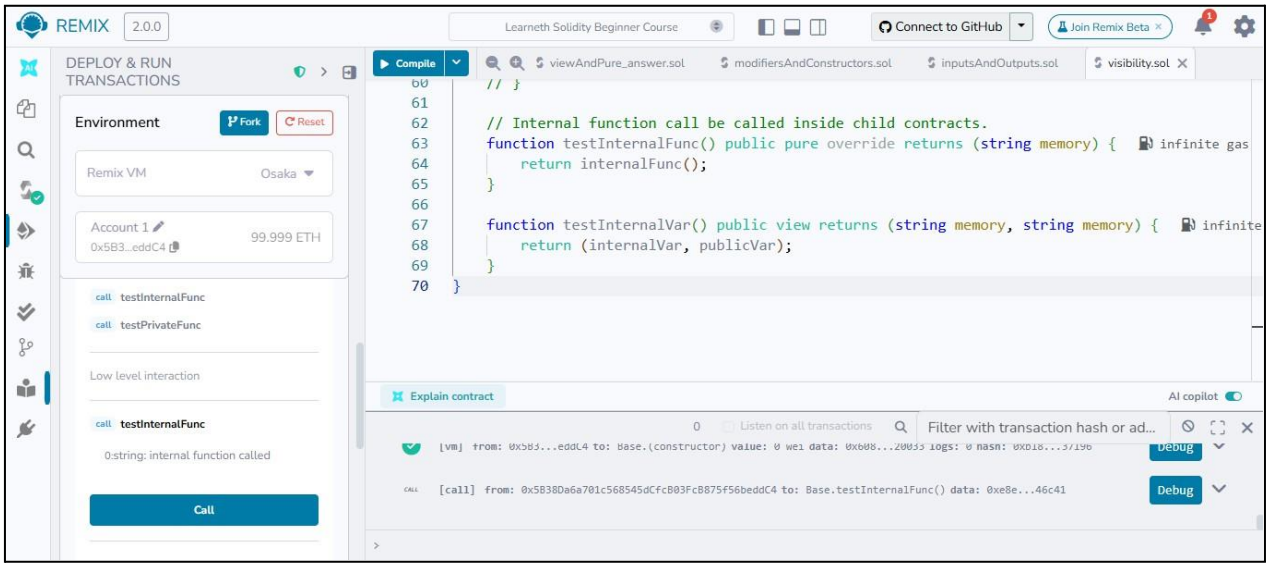




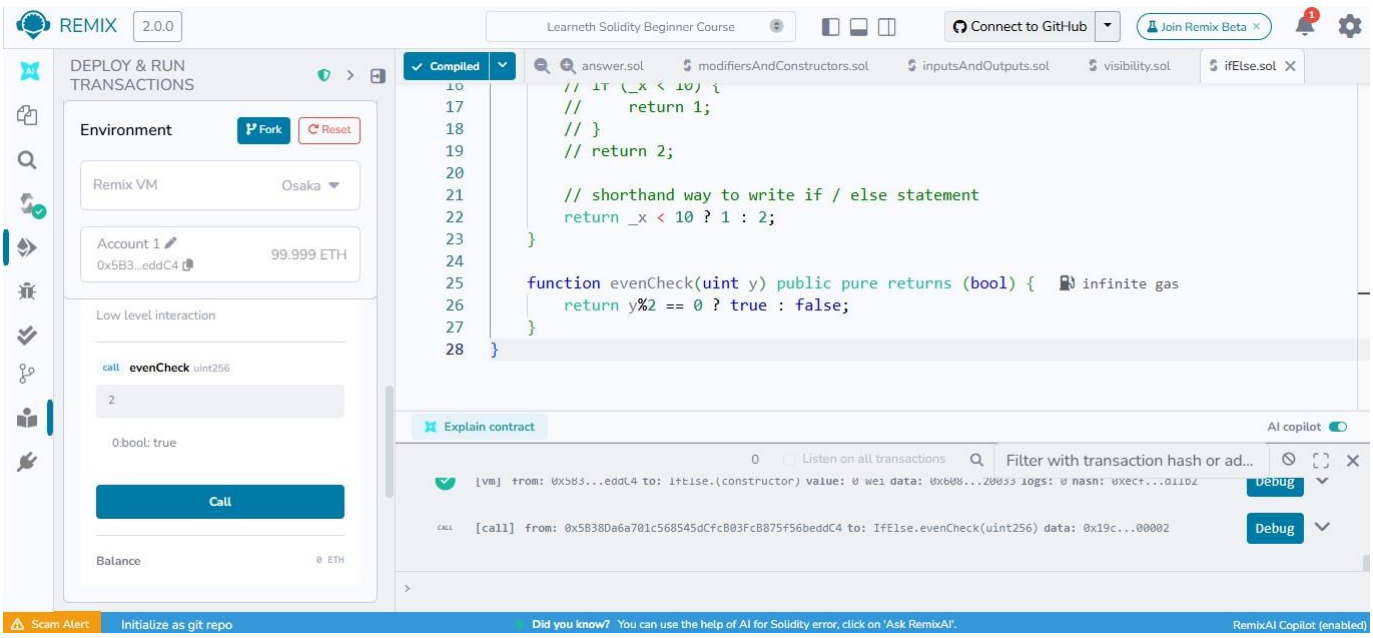
Tutorial 5.4 Functions - Inputs and Outputs

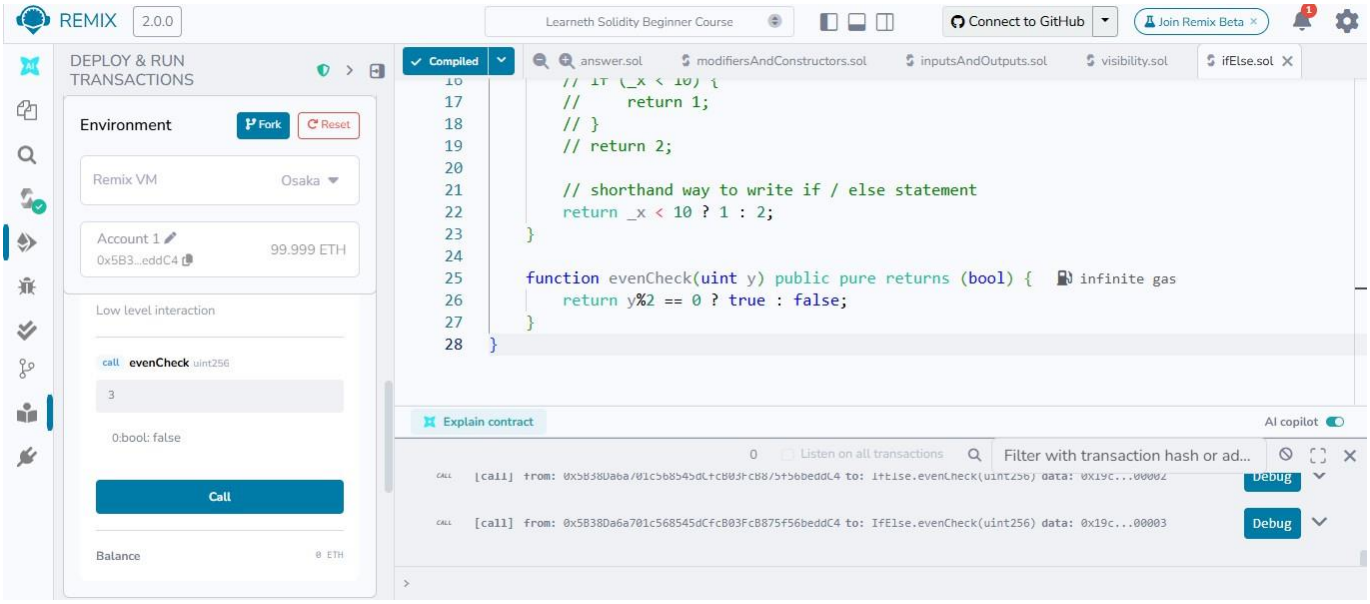


Tutorial 6. Visibility

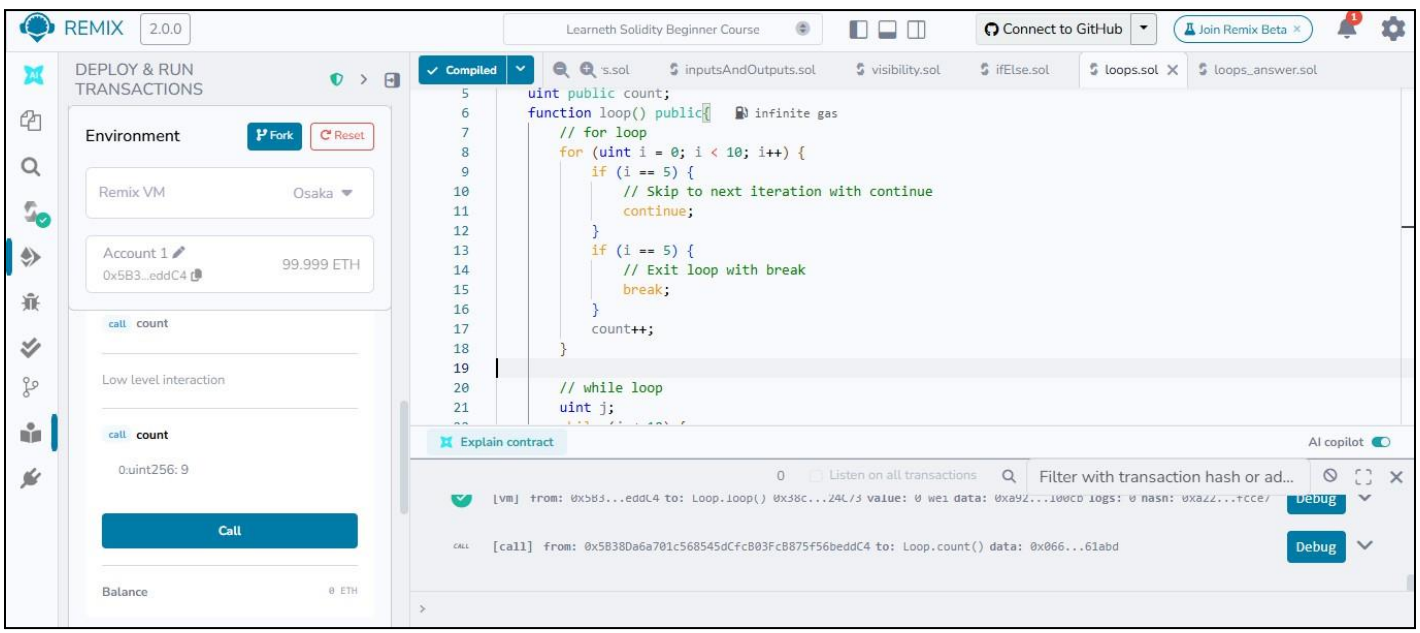


Tutorial 7.1 Control Flow - If/Else

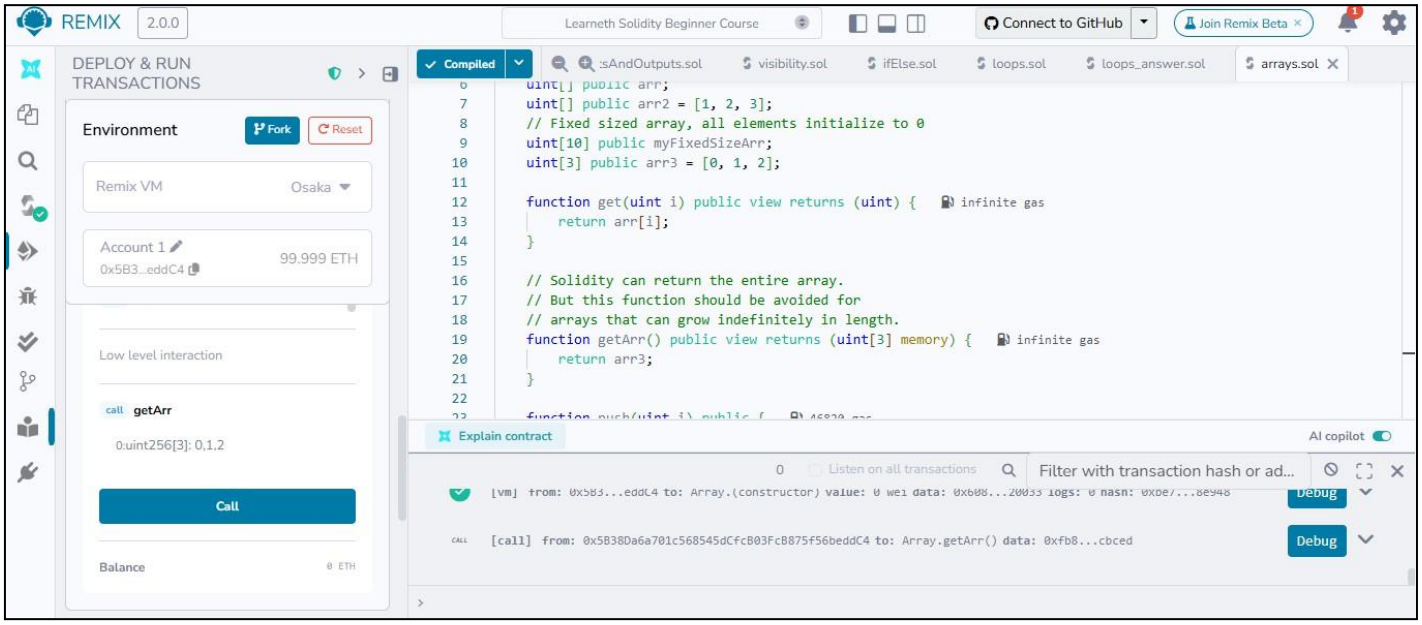




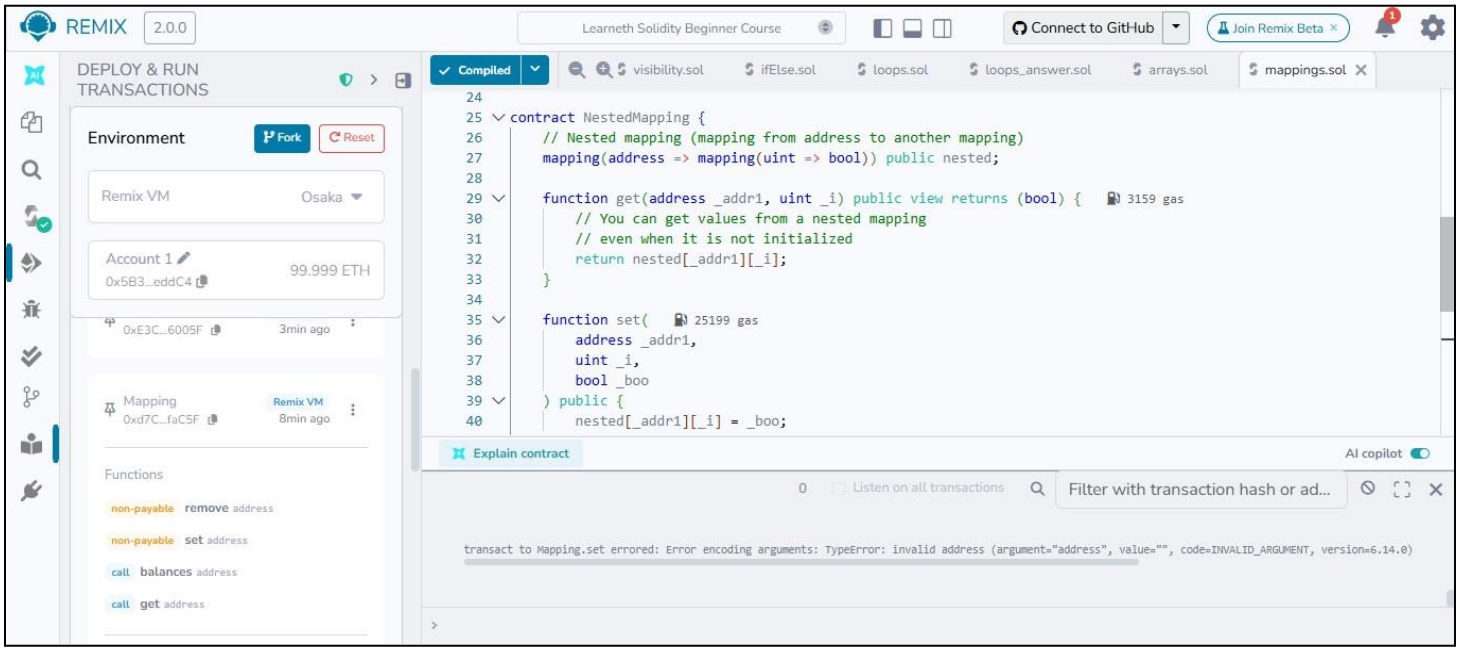
Tutorial 7.2 Control Flow - Loops



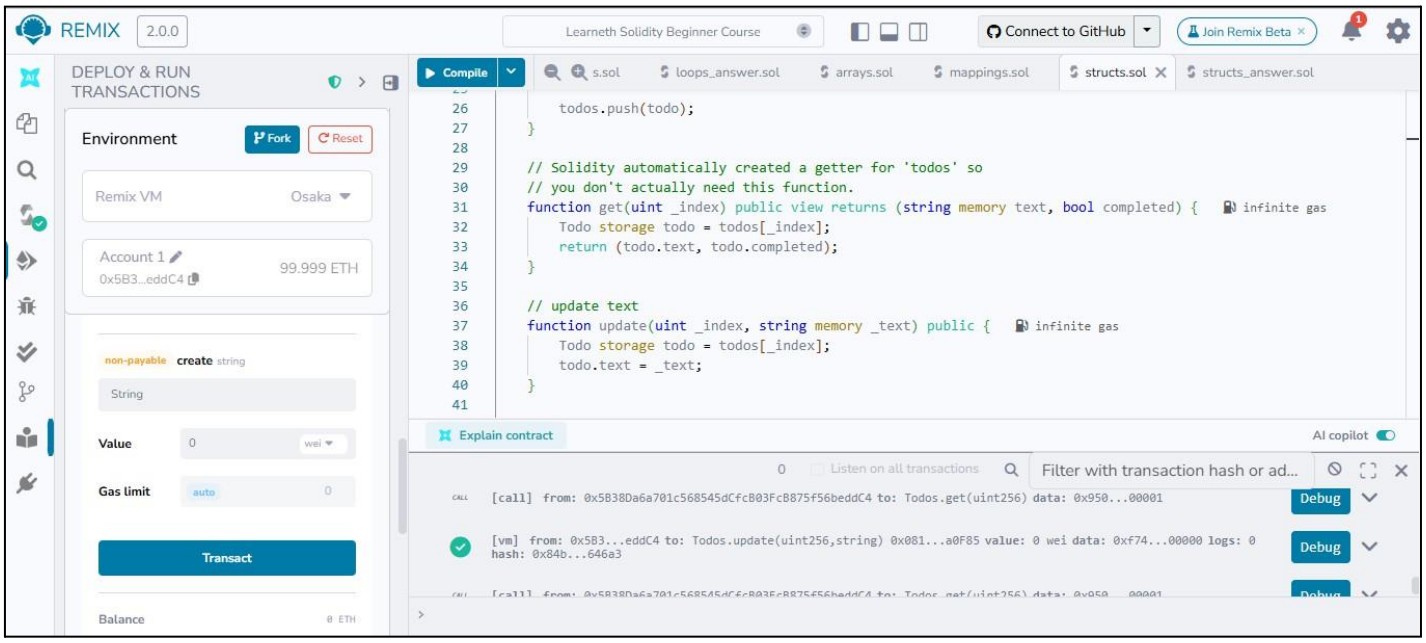
Tutorial 8.1 Data Structures - Arrays

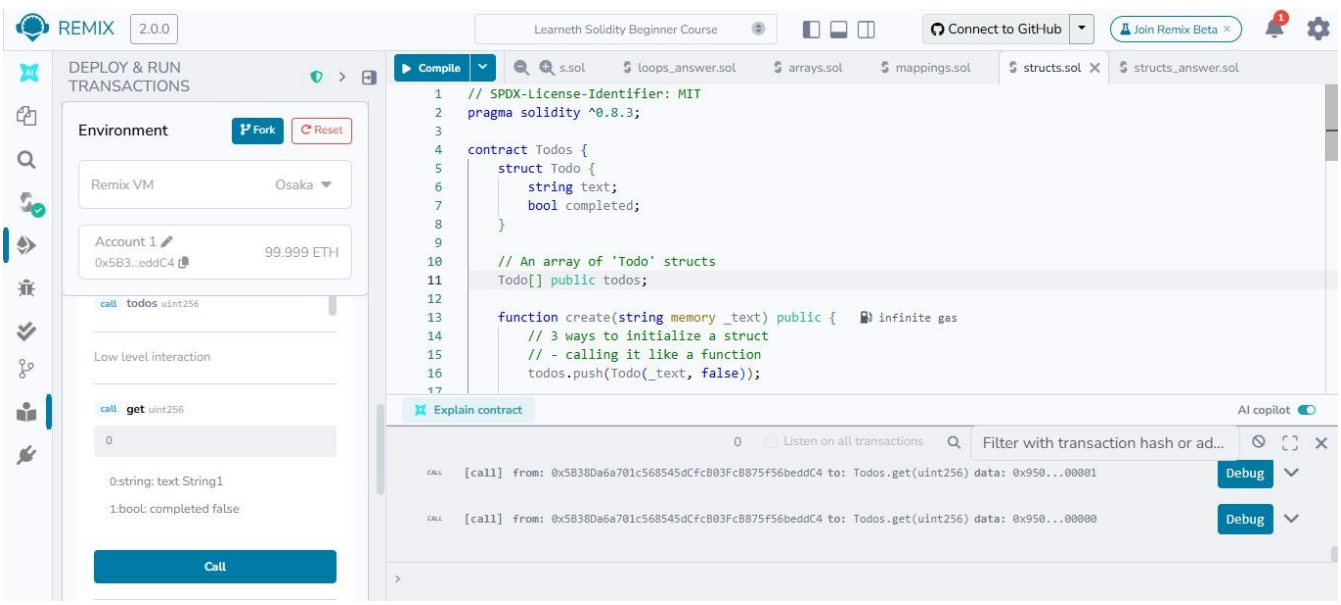


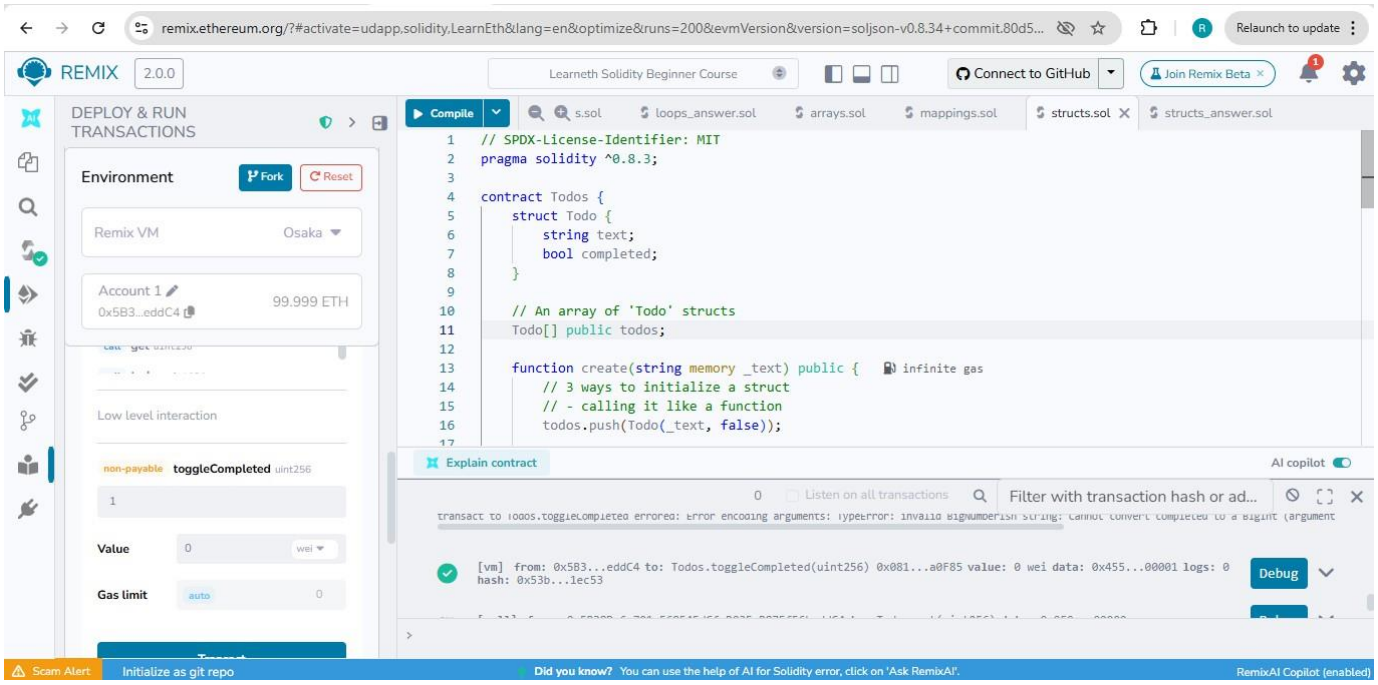
Tutorial 8.2 Data Structures - Mappings

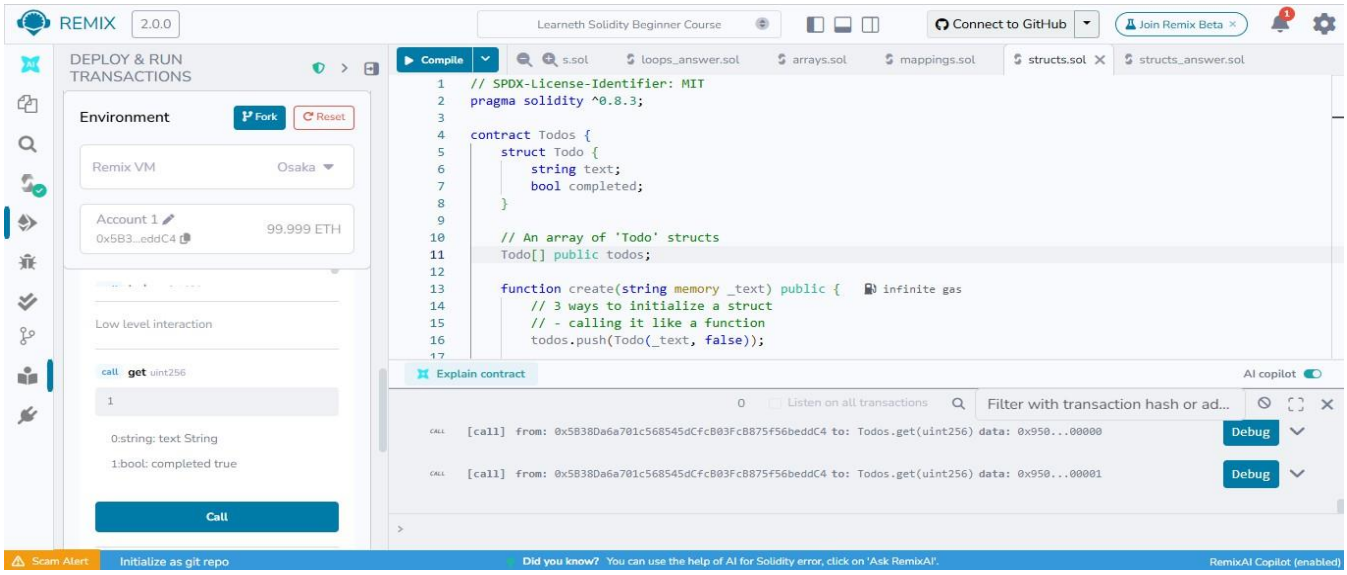


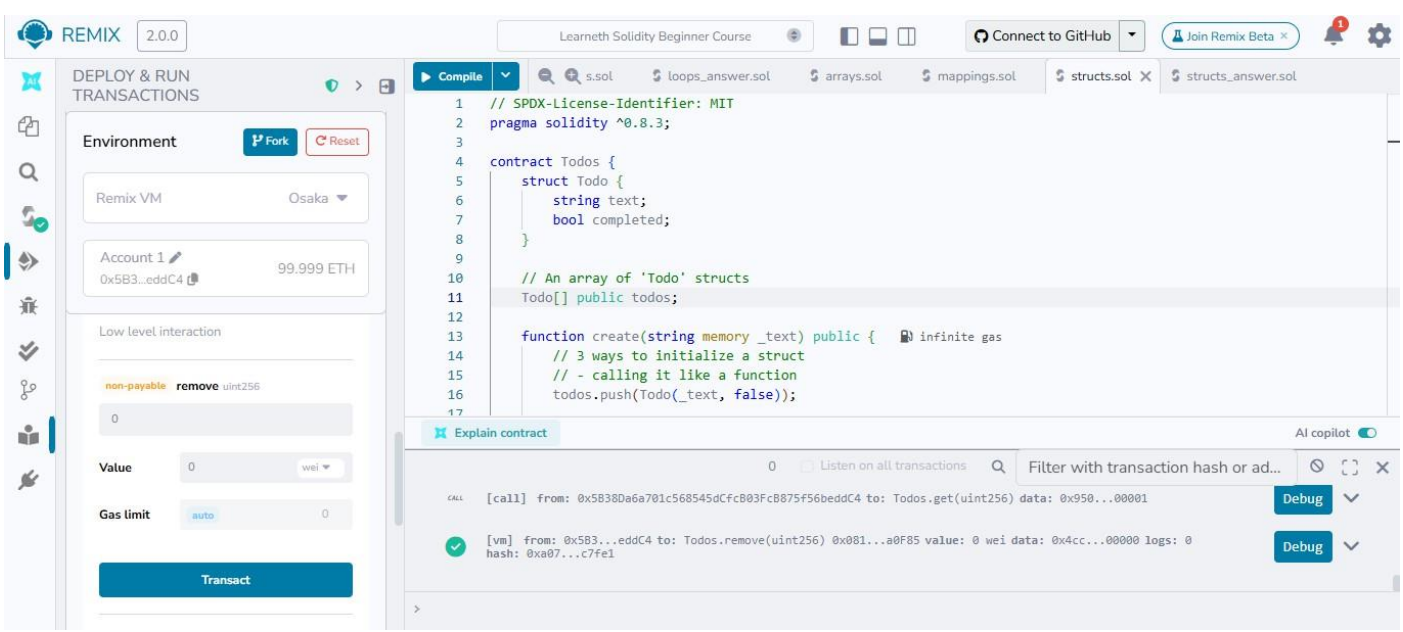
8.3 Data Structures - Structs

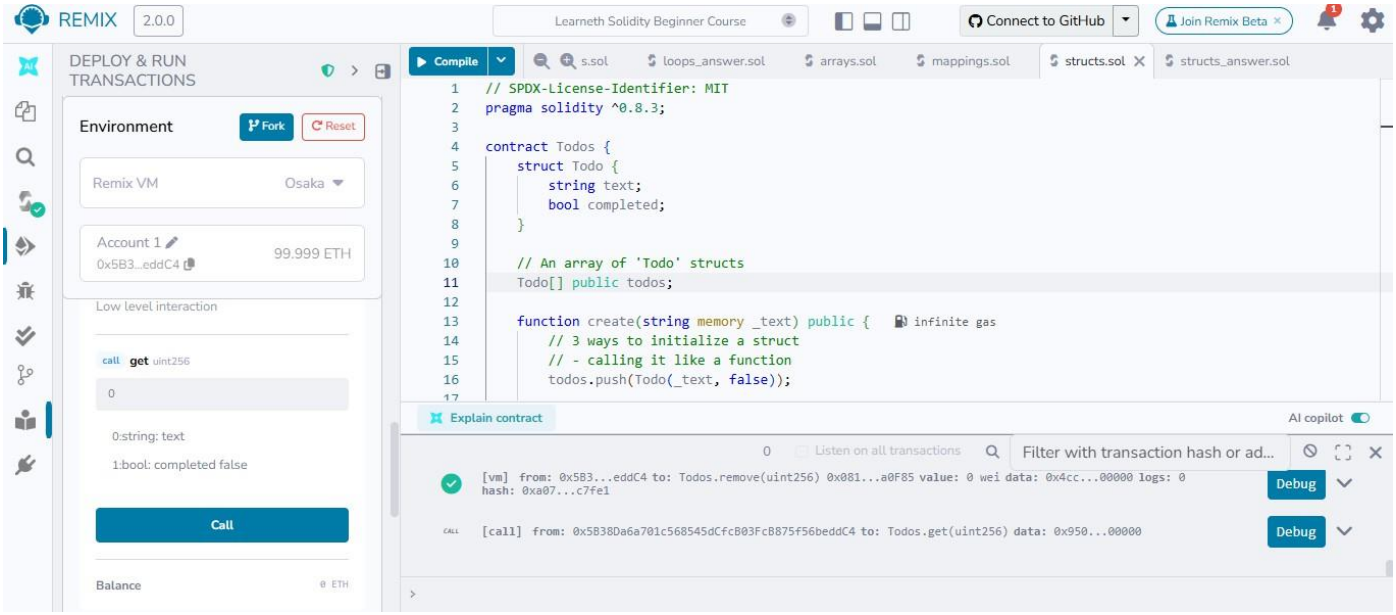




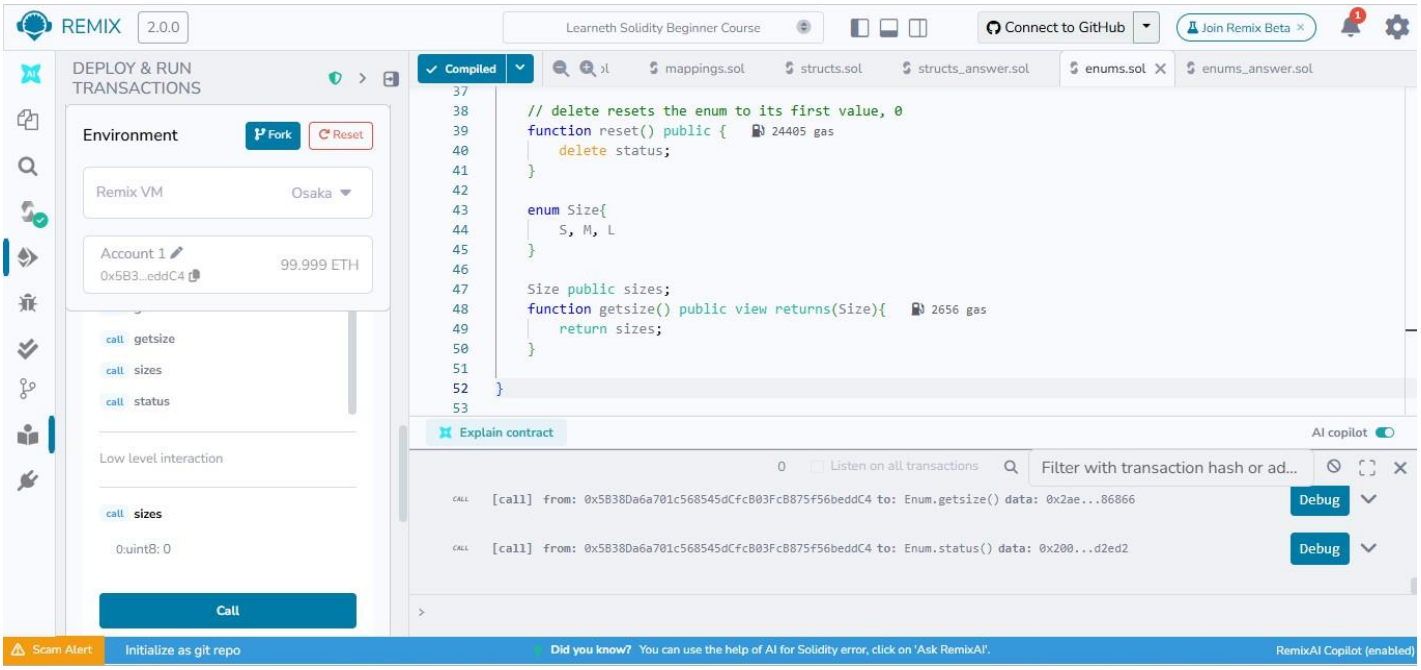

remix.ethereum.org/?#activate=udapp,solidity,LearnEth&lang=en&optimize&runs=200&evmVersion&version=soljson-v0.8.34+commit.80d5...
Relaunch to update
REMIX 2.0.0
Learneth Solidity Beginner Course
Connect to GitHub
Join Remix Beta
DEPLOY & RUN TRANSACTIONS
Compile
s.sol
loops_answer.sol
arrays.sol
mappings.sol
structs.sol
structs_answer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Todos {
struct Todo {
string text;
bool completed;
}
// An array of 'Todo' structs
Todo[] public todos;
function create(string memory _text) public {
infinite gas
// 3 ways to initialize a struct
// - calling it like a function
todos.push(Todo(_text, false));
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Low level interaction
non-payable toggleCompleted uint256
1
Value
0
wei
Gas limit
auto
0
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[vm] from: 0x5B3...eddC4 to: Todos.toggleCompleted(uint256) 0x081...a0F85 value: 0 wei data: 0x455...00001 logs: 0 hash: 0x53b...1ec53
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.
RemixAI Copilot (enabled)


REMIX 2.0.0
Learneth Solidity Beginner Course
Connect to GitHub
Join Remix Beta
DEPLOY & RUN TRANSACTIONS
Compile
s.sol
loops_answer.sol
arrays.sol
mappings.sol
structs.sol
structs_answer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Todos {
struct Todo {
string text;
bool completed;
}
// An array of 'Todo' structs
Todo[] public todos;
function create(string memory _text) public {
infinite gas
// 3 ways to initialize a struct
// - calling it like a function
todos.push(Todo(_text, false));
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Low level interaction
call get uint256
1
0:string: text String
1:bool: completed true
Call
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Todos.get(uint256) data: 0x950...00000
Debug
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Todos.get(uint256) data: 0x950...00001
Debug
Scam Alert
Initialize as git repo
Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.
RemixAI Copilot (enabled)

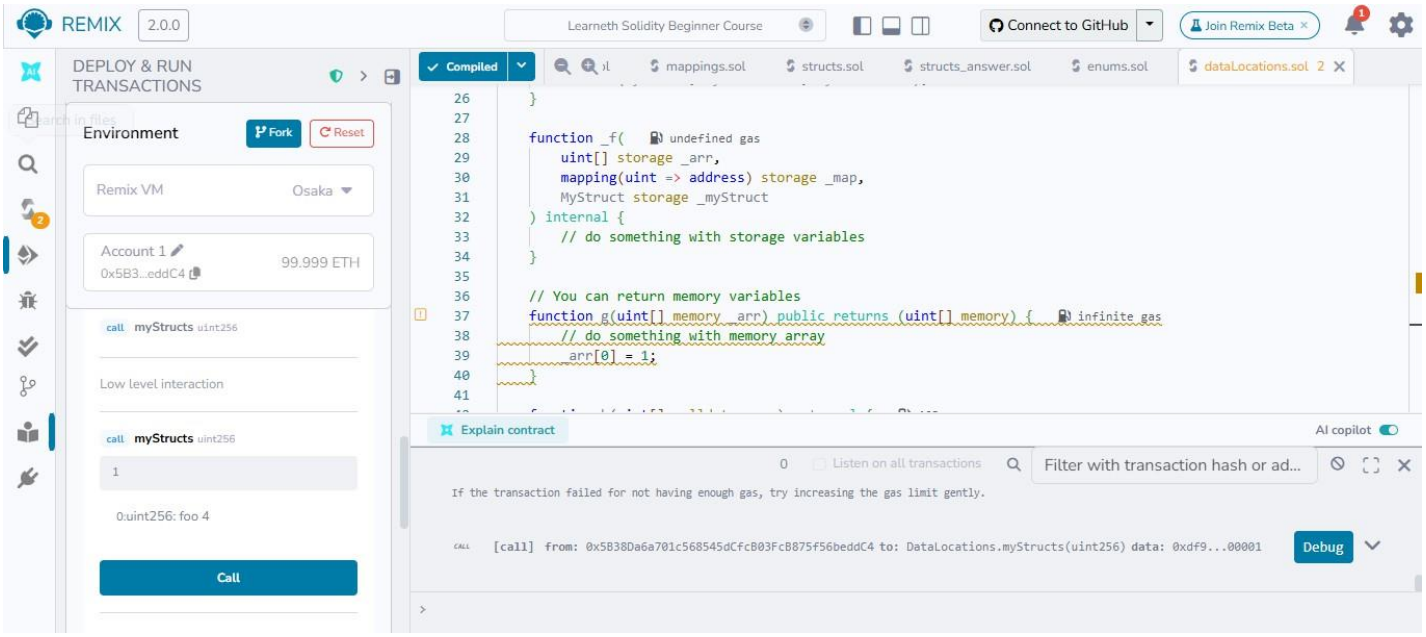

REMIX 2.0.0
Learneth Solidity Beginner Course
Connect to GitHub
Join Remix Beta
DEPLOY & RUN TRANSACTIONS
Compile
s.sol
loops_answer.sol
arrays.sol
mappings.sol
structs.sol
structs_answer.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Todos {
struct Todo {
string text;
bool completed;
}
// An array of 'Todo' structs
Todo[] public todos;
function create(string memory _text) public {
infinite gas
// 3 ways to initialize a struct
// - calling it like a function
todos.push(Todo(_text, false));
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Low level interaction
non-payable remove uint256
0
Value
0
wei
Gas limit
auto
0
Transact
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Todos.get(uint256) data: 0x950...00001
Debug
[vm] from: 0x5B3...eddC4 to: Todos.remove(uint256) 0x081...a0F85 value: 0 wei data: 0x4cc...00000 logs: 0 hash: 0xa07...c7fe1
Debug



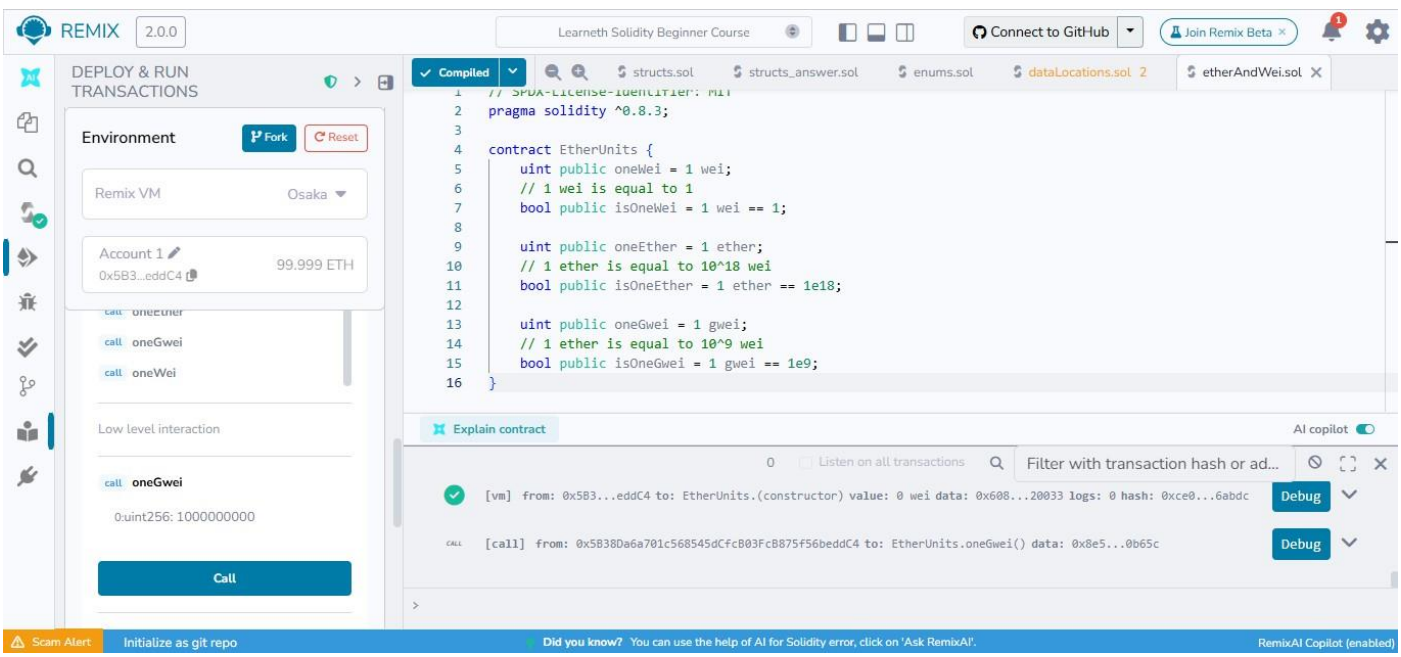
8.4 Data Structures - Enums



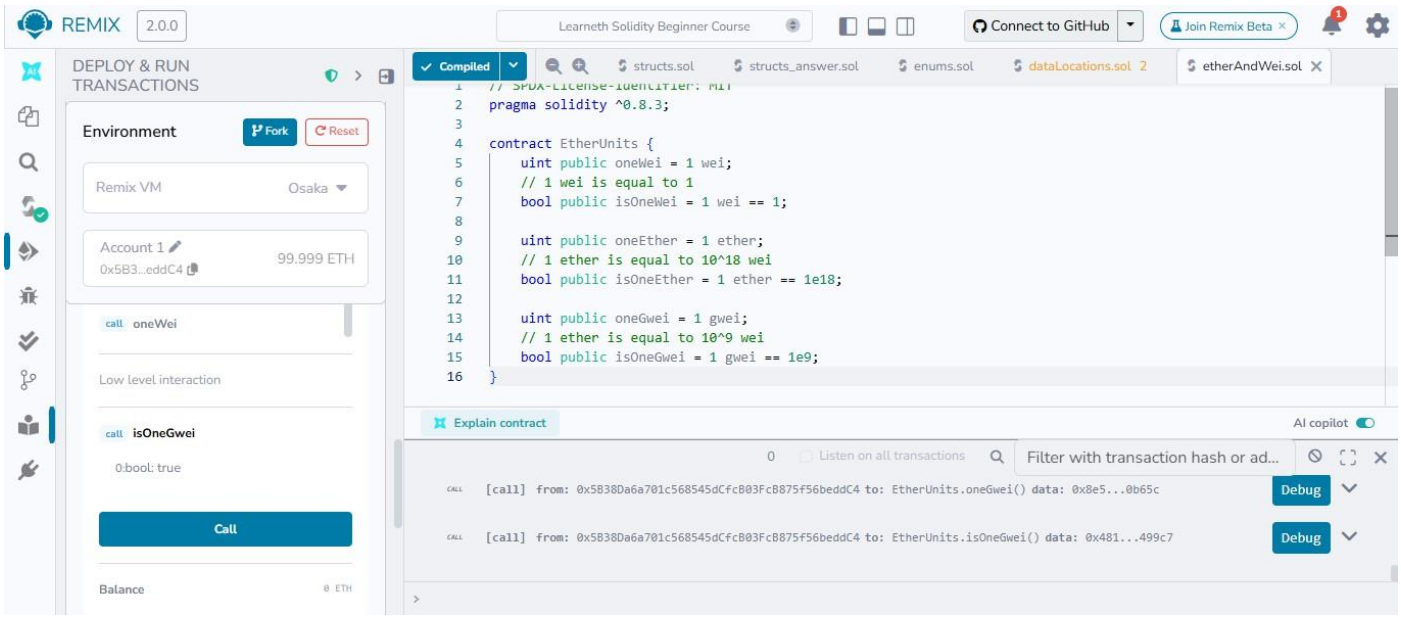
9. Data Locations

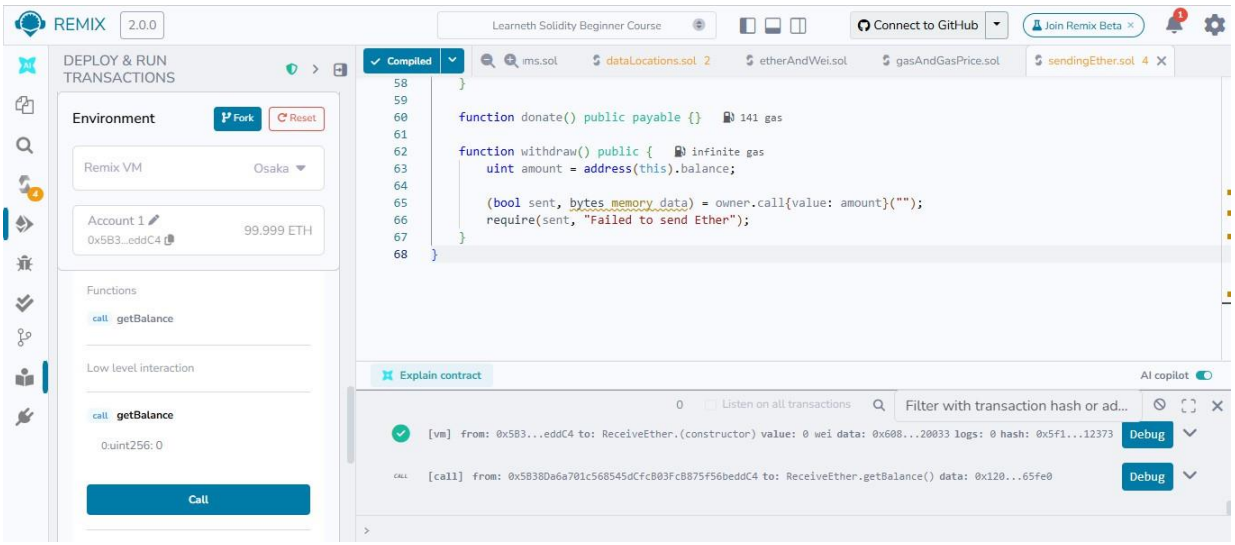


10.1 Transactions - Ether and Wei

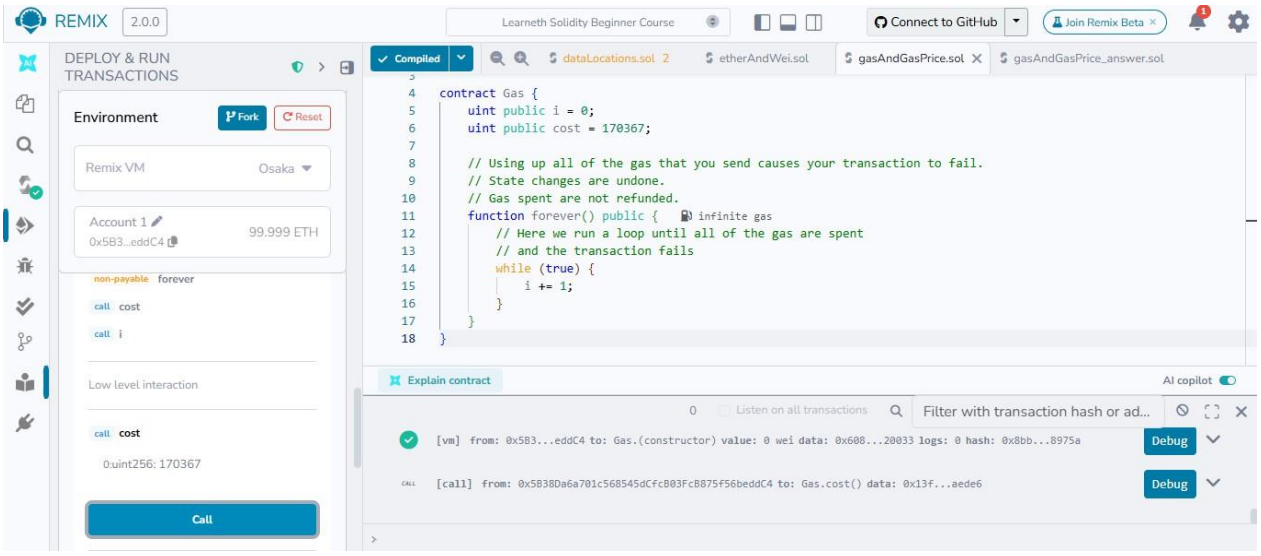


10.2 Transactions - Gas and Gas Price





10.3 Transactions - Sending Ether



Conclusion:

This experiment provided an in-depth exploration of Solidity programming through structured practical implementation using the Remix IDE. Fundamental concepts—including data types, variable classifications, function definitions, visibility specifiers, modifiers, constructors, control flow mechanisms, data structures, and transaction management—were systematically implemented and evaluated.

The process of designing, compiling, and deploying smart contracts on the Remix Virtual Machine (VM) enabled a comprehensive understanding of smart contract architecture and blockchain execution mechanisms. The practical exposure strengthened conceptual clarity and technical proficiency in Ethereum-based development.